

05–Spring全家桶

Spring 全家桶面试题

1. Spring 核心

问：什么是 IOC 和 DI?

答：– IOC（控制反转）：对象创建和依赖管理交给 Spring 容器，而非程序员 new – DI（依赖注入）：容器在运行时注入依赖，方式有构造器注入、Setter 注入、字段注入（@Autowired）

好处：解耦、便于测试、统一管理 Bean 生命周期。

问：依赖倒置、依赖注入、控制反转分别是什么？

答：

概念	含义
依赖倒置（DIP）	高层不依赖低层，都依赖抽象（接口）
控制反转（IoC）	对象创建和依赖管理的控制权交给容器
依赖注入（DI）	IoC 的实现方式，容器在运行时注入依赖

关系：DIP 是设计原则，IoC 是思想，DI 是具体手段。

问：Spring 核心思想？用了哪些设计模式？

答：核心：IoC 容器 + AOP 切面 + 简化企业开发（事务、MVC、数据访问）

常见设计模式：

模式	体现
工厂	BeanFactory、FactoryBean
单例	Bean 默认 singleton
代理	AOP（JDK/CGLIB）
模板	JdbcTemplate、RestTemplate
观察者	ApplicationEvent / Listener
适配器	HandlerAdapter、Spring MVC
装饰器	TransactionAwareCacheDecorator
策略	多种 BeanPostProcessor、Resource 加载

问：Spring Bean 的生命周期？

答：

```
1. 实例化（构造器）
2. 属性注入（@Autowired）
3. Aware 接口回调（BeanNameAware 等）
4. BeanPostProcessor.postProcessBeforeInitialization
5. 初始化
   ├── @PostConstruct
   ├── InitializingBean.afterPropertiesSet()
   └── 自定义 init-method
6. BeanPostProcessor.postProcessAfterInitialization
7. Bean 可用
8. 销毁
   ├── @PreDestroy
   ├── DisposableBean.destroy()
   └── 自定义 destroy-method
```

问：Spring 如何解决循环依赖？

答：仅 单例 + 属性注入 可解决，通过 三级缓存：

缓存	内容
一级 singletonObjects	完整 Bean
二级 earlySingletonObjects	早期 Bean（已实例化未初始化）
三级 singletonFactories	ObjectFactory，用于生成早期引用

流程：A 依赖 B，B 依赖 A → A 实例化后放入三级 → 创建 B → B 需要 A 从三级获取早期 A → B 完成 → A 注入 B → A 完成。

无法解决：构造器注入循环依赖、多例 Bean。

问：Spring AOP 原理？

答：面向切面编程，将日志、事务、权限等横切逻辑与业务分离。

实现方式： – 有接口：JDK 动态代理（基于接口反射） – 无接口：CGLIB（生成子类，继承目标类）

核心概念： – 切面（Aspect）：横切逻辑的模块 – 切点（Pointcut）：匹配连接点的表达式 – 通知（Advice）：Before、After、Around、AfterReturning、AfterThrowing – 连接点（JoinPoint）：可被拦截的方法

问：Spring 事务传播行为？

答：

传播行为	说明
REQUIRED (默认)	有事务加入，无则新建
REQUIRES_NEW	总是新建，挂起当前事务
NESTED	嵌套事务，外层回滚则内层也回滚
SUPPORTS	有则加入，无则非事务执行
NOT_SUPPORTED	非事务执行，挂起当前事务
MANDATORY	必须在事务中，否则抛异常
NEVER	不能在事务中，否则抛异常

问：Spring 事务失效场景？

答：1. 方法非 `public` 2. 同类自调用（绕过代理） 3. 异常被 `catch` 未抛出 4. 抛出的异常类型不匹配（默认只回滚 `RuntimeException`） 5. 数据库引擎不支持事务（MyISAM） 6. 未被 Spring 管理（未加 `@Service` 等）

问：Spring 常用注解有哪些？

答：

类别	注解
组件注册	<code>@Component</code> 、 <code>@Service</code> 、 <code>@Repository</code> 、 <code>@Controller</code> 、 <code>@RestController</code>
依赖注入	<code>@Autowired</code> 、 <code>@Resource</code> 、 <code>@Qualifier</code> 、 <code>@Value</code>
配置	<code>@Configuration</code> 、 <code>@Bean</code> 、 <code>@ComponentScan</code> 、 <code>@Import</code>
AOP/事务	<code>@Aspect</code> 、 <code>@Transactional</code> 、 <code>@EnableAspectJAutoProxy</code>
Web	<code>@RequestMapping</code> 、 <code>@GetMapping</code> 、 <code>@RequestBody</code> 、 <code>@PathVariable</code>
Boot	<code>@SpringBootApplication</code> 、 <code>@ConditionalOnXxx</code> 、 <code>@ConfigurationProperties</code>
生命周期	<code>@PostConstruct</code> 、 <code>@PreDestroy</code> 、 <code>@Scope</code>

2. Spring MVC

问：Spring MVC 请求处理流程？

答：

1. `DispatcherServlet` 接收请求
2. `HandlerMapping` 找到 `Handler`
3. `HandlerAdapter` 执行 `Handler` (`Controller`)
4. 返回 `ModelAndView` 或 `@ResponseBody`
5. `ViewResolver` 解析视图 (REST 直接写响应)
6. 渲染视图返回客户端

问: `@RestController` 和 `@Controller` 区别?

答: - `@Controller`: 返回视图名, 配合 `@ResponseBody` 返回 JSON - `@RestController` = `@Controller` + `@ResponseBody`, 方法直接返回 JSON

问: `Filter` 和 `Interceptor` 的区别?

答:

对比	Filter	Interceptor
规范	Servlet 规范	Spring 提供
作用范围	所有请求	Spring MVC 请求
依赖	不依赖 Spring	依赖 Spring 容器
执行时机	<code>DispatcherServlet</code> 之前	<code>DispatcherServlet</code> 之后

Filter 典型: 编码、CORS、鉴权 Token 校验。 **Interceptor 典型:** 登录校验、日志、权限 (可访问 Handler 信息)。

问: `HandlerMapping` 和 `HandlerAdapter` 是什么?

答:

组件	作用
<code>HandlerMapping</code>	根据 URL/注解将请求 映射到 Handler (<code>Controller</code> 方法), 返回 <code>HandlerExecutionChain</code> (含拦截器)
<code>HandlerAdapter</code>	适配并调用 <code>Handler</code> , 统一处理参数绑定、校验、返回值转换

为什么需要 Adapter: `Controller` 实现方式多样 (`@RequestMapping`、`HttpRequestHandler` 等), `Adapter` 屏蔽差异, `DispatcherServlet` 只依赖统一接口。

3. Spring Boot

问: Spring Boot 自动配置原理?

答:

```

@SpringBootApplication
├─ @SpringBootConfiguration
├─ @EnableAutoConfiguration
│   └─ @Import(AutoConfigurationImportSelector)
│       └─ 读取 META-INF/spring.factories (或 AutoConfiguration.imports)
│           └─ 加载自动配置类
│               └─ @ConditionalOnXxx 条件装配
└─ @ComponentScan

```

满足条件（如 classpath 有某个类）才生效对应配置。

问：为什么用 Spring Boot? 约定大于配置怎么理解?

答：优势： – 自动配置，减少 XML/样板代码 – Starter 一键集成常用框架 – 内嵌 Tomcat/Jetty, jar 直接运行 – Actuator 监控、外部化配置

约定大于配置： – 默认内嵌 Tomcat、默认端口 8080 – @SpringBootApplication 扫描同级及子包 – application.yml 统一配置 – 引入 spring-boot-starter-web 自动配 MVC

Override： 需要时用 @Configuration + @Bean 或 application.yml 覆盖默认。

问：Spring Boot 常用 Starter 有哪些?

答：

Starter	作用
spring-boot-starter-web	Web + Tomcat + Jackson
spring-boot-starter-data-jpa	JPA + Hibernate
spring-boot-starter-redis	Lettuce/Redis
spring-boot-starter-amqp	RabbitMQ
spring-boot-starter-test	JUnit + MockMvc
spring-boot-starter-actuator	健康检查、指标

自定义 Starter： autoconfigure 模块 + @ConditionalOnXxx + AutoConfiguration.imports 注册。

问：Spring Boot 启动流程?

答： 1. 创建 SpringApplication 2. 准备 Environment 3. 打印 Banner 4. 创建 ApplicationContext 5. refresh() 刷新容器（核心） 6. 启动内嵌 Tomcat/Jetty/Undertow 7. 执行 ApplicationRunner / CommandLineRunner

4. MyBatis

问：#{ } 和 \${ } 的区别？

答：

对比	#{ }	\${ }
方式	预编译占位符	字符串拼接
安全性	防 SQL 注入	有注入风险
场景	参数值	表名、列名、ORDER BY

问：MyBatis 一级缓存和二级缓存？

答： – 一级缓存：SqlSession 级别，默认开启，同一 Session 相同查询走缓存 – 二级缓存：Mapper 级别，需 <cache/> 开启，跨 SqlSession，注意序列化和脏读

清除：增删改、commit、close、clearCache。

问：MyBatis 和 JDBC 相比有什么优点？

答：

对比	JDBC	MyBatis
SQL	硬编码在 Java	XML/注解分离
参数	手动 PreparedStatement	#{ } 自动预编译
结果映射	手动 ResultSet 遍历	自动映射 POJO
维护	改动大	SQL 独立维护
缓存	无	一级/二级缓存

MyBatis 设计模式：建造者 (SqlSessionFactoryBuilder)、工厂 (SqlSessionFactory)、代理 (Mapper 接口 JDK 动态代理)、模板 (SqlSession)。

问：MyBatis-Plus 和 MyBatis 的区别？

答：

对比	MyBatis	MyBatis-Plus
CRUD	手写 SQL	内置 BaseMapper 通用 CRUD
分页	插件自行配置	Pagination 插件开箱即用
条件	XML 动态 SQL	LambdaQueryWrapper
定位	ORM 框架	MyBatis 增强工具，无侵入

5. Spring Cloud

问：Spring Cloud 常用组件？

答：

功能	组件
注册中心	Nacos、Eureka
配置中心	Nacos Config、Apollo
网关	Spring Cloud Gateway
负载均衡	LoadBalancer
远程调用	OpenFeign
熔断限流	Sentinel、Resilience4j
分布式事务	Seata

问：Spring Cloud 和 Spring Boot 的区别？

答：

对比	Spring Boot	Spring Cloud
定位	快速构建单应用	微服务治理工具集
关系	基础	基于 Boot，提供分布式能力
内容	自动配置、Starter	注册发现、配置中心、网关、熔断

Boot 解决「怎么快速开发一个应用」，Cloud 解决「多个应用怎么协作」。

问：负载均衡算法有哪些？什么是服务熔断和降级？

答：负载均衡：

算法	说明
轮询	依次分配
随机	随机选择
加权轮询/随机	按权重
最少连接	连接数最少优先
一致性 Hash	同一 key 固定节点（会话保持）

熔断（Circuit Breaker）：下游故障率超阈值 → 快速失败，不再调用，避免雪崩（Sentinel、Hystrix）。

降级（Fallback）：非核心功能返回兜底数据或默认值，保证核心链路可用。

6. 三级缓存详解

问：Spring 三级缓存分别是什么？为什么需要三级？

答：

缓存名	变量	存储内容
一级	singletonObjects	完全初始化好的 Bean
二级	earlySingletonObjects	早期暴露的 Bean（已实例化，未填充属性）
三级	singletonFactories	ObjectFactory 工厂，调用 <code>getEarlyBeanReference()</code>

为什么需要三级： – 二级存早期引用足够解决普通循环依赖 – 三级是为了 **AOP 代理**：若 Bean 需要代理，三级工厂的 `getEarlyBeanReference()` 会返回 **代理对象** 而非原始对象 – 保证注入的是最终代理对象，避免 A 注入的是 B 的原始对象而其他地方用的是 B 的代理

流程 (A→B→A)： 1. 创建 A，实例化后放三级缓存 (ObjectFactory) 2. 填充 A 的属性，发现需要 B 3. 创建 B，实例化后放三级 4. B 填充属性需要 A，从三级获取 A 的早期引用（可能是代理） 5. B 完成，注入 A 6. A 完成，从三级移除，放入一级

问：哪些循环依赖 Spring 无法解决？

答： 1. **构造器注入**：实例化阶段就需要依赖，无法提前暴露 2. **多例 Bean (prototype)**：每次 new，不缓存 3. **@Async 等后置处理**导致的复杂依赖 4. 三个及以上 Bean 循环依赖 (A→B→C→A) 理论上可解，但构造器注入仍不行

解决构造器循环：改用 `@Lazy` 延迟注入，或改 setter/字段注入。

7. AOP 深入

问：Spring AOP 何时用 JDK 动态代理，何时用 CGLIB？

答：

条件	选择
目标类实现了接口	默认 JDK 动态代理
目标类无接口	CGLIB
强制 CGLIB	<code>@EnableAspectJAutoProxy(proxyTargetClass = true)</code>

JDK 动态代理：基于接口，生成 `$Proxy` 类，只能代理接口方法。 **CGLIB**：生成目标类子类，不能代理 `final` 方法和类。

Spring Boot 2.x+ 默认 `proxyTargetClass = true`，统一用 CGLIB。

问：静态代理和动态代理的区别？ AOP 常用注解？

答：

对比	静态代理	动态代理
生成时机	编译期手写代理类	运行期生成
灵活性	差，每个目标需一个类	好，统一 Handler
Spring AOP	可以但不采用	JDK / CGLIB

AOP 注解：

注解	作用
@Aspect	声明切面类
@Pointcut	切点表达式
@Before / @After	前置/后置
@Around	环绕（可控制是否 proceed）
@AfterReturning / @AfterThrowing	返回/异常通知

问：JDK 动态代理和 CGLIB 的性能和区别？

答：

对比	JDK 动态代理	CGLIB
原理	反射 InvocationHandler	ASM 字节码生成子类
要求	必须有接口	不能代理 final
创建速度	快	慢（首次生成字节码）
调用速度	慢（反射）	快（方法调用）
Spring 选择	有接口优先	无接口 / 强制

JDK 8+ 反射优化后，差距缩小。Spring 默认 CGLIB 简化配置。

8. @Transactional 原理

问：@Transactional 的实现原理？

答：基于 AOP 动态代理：1. @EnableTransactionManagement 导入 TransactionManagementConfigurationSelector 2. 注册 InfrastructureAdvisorAutoProxyCreator（BeanPostProcessor） 3. 匹配 @Transactional 方法，创建代理对象 4. 调用时走 TransactionInterceptor

TransactionInterceptor 流程：

1. 获取 `TransactionManager`
2. 根据 `@Transactional` 属性创建 `TransactionDefinition`
3. `transactionManager.getTransaction()` 开启事务
4. 执行业务方法 (`invocation.proceed()`)
5. 无异常 → `commit`
6. 有异常 → 判断 `rollbackFor` → `rollback`

底层: `JDBC Connection.setAutoCommit(false)` + `commit/rollback`, 或 JTA 分布式事务。

问: `@Transactional` 同类自调用为什么失效? 怎么解决?

答: Spring AOP 代理的是 外部调用。同类内 `this.method()` 调用的是原始对象, 不经过代理。

解决: 1. 注入自身代理: `@Autowired self; self.method();` 2. 拆分到另一个 Service 3. 使用 `AopContext.currentProxy()` (需 `@EnableAspectJAutoProxy(exposeProxy = true)`) 4. 改用 AspectJ 编译织入 (少见)

9. Spring Boot Starter

问: 如何自定义 Spring Boot Starter?

答: 命名规范: `xxx-spring-boot-starter` (官方) 或 `spring-boot-starter-xxx` (第三方)

步骤: 1. 创建 `autoconfigure` 模块, 写 `@Configuration + @ConditionalOnXxx` 2. 写 `xxxProperties` 绑定 `@ConfigurationProperties` 3. 在 `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` 注册自动配置类 4. 创建 `starter` 模块, 依赖 `autoconfigure`, 供用户引入

常用条件注解: - `@ConditionalOnClass`: classpath 有某类 - `@ConditionalOnMissingBean`: 容器无某 Bean 时才创建 - `@ConditionalOnProperty`: 配置项匹配

问: Spring Boot 的 SPI 机制与 Spring 有什么区别?

答: Spring Boot 2.7+ 用 `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` 替代 `spring.factories` 中的自动配置。

`spring.factories` 仍用于 `ApplicationContextInitializer`、`Listener` 等扩展。

10. Bean 作用域

问: Spring Bean 有哪些作用域?

答:

作用域	说明	使用场景
singleton（默认）	容器中唯一实例	无状态 Service、DAO
prototype	每次获取新建	有状态 Bean
request	每个 HTTP 请求一个	Web 应用
session	每个 HTTP Session 一个	用户会话数据
application	ServletContext 级别	全局 Web 数据

注意：singleton Bean 注入 prototype Bean 时，prototype 只在 singleton 创建时实例化一次。解决：@Scope(prototype) + @Lazy 或 ObjectFactory / Provider。

11. 其他深入

问：@Autowired 和 @Resource 的区别？

答：

对比	@Autowired	@Resource
来源	Spring	JSR-250 (Java)
装配	默认 byType	默认 byName
必需	required=true 默认	默认必需
属性	无 name	有 name 属性

问：Spring 中 BeanFactory 和 FactoryBean 的区别？

答：– BeanFactory：IOC 容器顶层接口，管理 Bean – FactoryBean：工厂 Bean，getObject() 返回的才是实际 Bean

典型：MyBatis 的 SqlSessionFactoryBean 生产 SqlSessionFactory。

获取 FactoryBean 本身：&beanName（加 & 前缀）。

问：Spring 有哪些扩展点？

答：

扩展点	时机	用途
BeanFactoryPostProcessor	Bean 实例化前	修改 BeanDefinition
BeanPostProcessor	Bean 实例化后	AOP 代理、属性修改
ApplicationContextInitializer	容器 refresh 前	修改 Environment
ImportSelector / ImportBeanDefinitionRegistrar	解析 @Import	动态注册 Bean
ApplicationListener	事件驱动	监听容器事件
HandlerInterceptor	MVC 请求	拦截器
@ControllerAdvice	全局	异常处理、数据绑定

问：Spring MVC 和 Spring WebFlux 的区别？

答：

对比	Spring MVC	WebFlux
模型	阻塞 Servlet	非阻塞 Reactive
底层	Tomcat/Jetty	Netty
编程	同步	Mono/Flux
场景	传统 CRUD	高并发 IO 密集

大厂追问

1. 三级缓存如果只用二级行不行？

普通无 AOP 的 Bean 可以。有 AOP 时，早期暴露的必须是代理对象，三级 ObjectFactory 负责在需要时生成代理，否则循环依赖中注入的是原始对象，导致不一致。

2. @Transactional 在 private 方法上有效吗？

无效。AOP 代理无法拦截 private 方法（子类无法 override），且 Spring 事务基于代理，private 方法不经过代理。

3. Spring Boot 如何内嵌 Tomcat？

SpringApplication.run() → 创建 ServletWebServerApplicationContext → onRefresh() 中 createWebServer() → 创建 TomcatServletWebServerFactory → 启动 Tomcat, DispatcherServlet 注册到 Tomcat。

4. NESTED 和 REQUIRES_NEW 区别?

REQUIRES_NEW: 完全独立事务, 外层回滚不影响内层。NESTED: 嵌套在外层事务中, 内层是 savepoint, 外层回滚则内层也回滚, 内层回滚不影响外层。

5. 如何排查 Spring 循环依赖?

启动报 `BeanCurrentlyInCreationException`, 看循环链 $A \rightarrow B \rightarrow A$ 。解决: 改 setter 注入、@Lazy、重构拆分依赖。

6. @Async 事务会生效吗?

@Async 方法在另一线程执行, 若通过代理调用且配置了 `TransactionManager`, 可生效; 同类自调用或线程切换导致事务上下文丢失则失效。

7. Spring Boot 如何开启事务?

@EnableTransactionManagement (Boot 自动开启) + Service 方法加 @Transactional + 引入 `spring-boot-starter-jdbc` 或 JPA 自动配 `DataSourceTransactionManager`。

8. OpenFeign 原理简述?

基于 JDK 动态代理 + Spring MVC 注解解析, 将 @FeignClient 接口方法转为 HTTP 请求, 配合 LoadBalancer 做服务发现和负载均衡。

9. singleton Bean 注入 prototype 怎么保证每次新建?

@Autowired ObjectFactory<T>、Provider<T>, 或 @Lookup 方法注入, 每次调用获取新实例。

10. Spring MVC 参数绑定流程?

HandlerAdapter 调用 `RequestMappingHandlerAdapter`, 通过 `HandlerMethodArgumentResolver` 链解析 @RequestBody、@PathVariable、@RequestParam 等, 底层用 `HttpMessageConverter` 反序列化 JSON。